

RustGraph: Robust Anomaly Detection in Dynamic Graphs by Jointly Learning Structural-Temporal Dependency

Jianhao Guo¹, Siliang Tang¹, Member, IEEE, Juncheng Li¹, Kaihang Pan¹, and Lingfei Wu²

Abstract—Dynamic graph-based data are ubiquitous in the real world, such as social networks, finance systems, and traffic flow. Fast and accurately detecting anomalies in these dynamic graphs is of vital importance. However, despite promising results the current anomaly detection methods have achieved, there are two major limitations when coping with dynamic graphs. The first limitation is that the topological structures and the temporal dynamics have been modeled separately, resulting in less expressive features for detection. The second limitation is that the models have been trained by unreliable noisy labels generated by random negative sampling, rendering it severely vulnerable to subtle perturbations. To overcome the above limitations, we propose RustGraph, a robust anomaly detection framework by jointly learning structural-temporal dependency in dynamic graphs. To this end, we design a variational graph auto-encoder with informative prior that simultaneously encodes both graph structural and temporal information. Then we introduce a fine-grained contrastive learning method to learn better node representations by utilizing the temporal consistency between two snapshots. Furthermore, we formulate the noisy label learning problem for anomaly detection in dynamic graph, and then propose a robust anomaly detector to improve the model performance by leveraging learned graph structure signal. Our extensive experiments on six real-world datasets demonstrate the proposed RustGraph method achieves state-of-the-art performance with an average of 3.64% improvement on AUC-ROC metric compared with all baselines. The codes are available at <https://github.com/aubreygjh/RustGraph>.

Index Terms—Anomaly detection, dynamic graphs, learning with noisy labels.

I. INTRODUCTION

DYNAMIC graphs, where the topological and attributive information of graph data can vary over time, are a prevalent and important data structure [1], [2]. Dynamic graphs widely exist in real-life scenarios, such as social networks [3], [4],

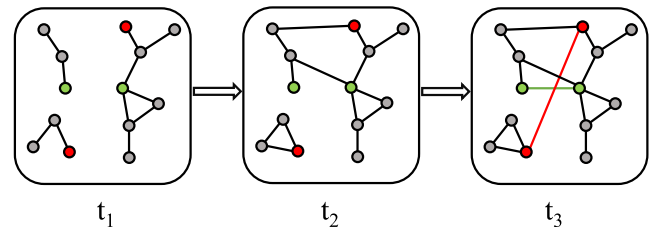


Fig. 1. Illustration that both structural and temporal information should be considered jointly. The green edge in t_3 tends to be normal because the structural connection between the green nodes is getting closer from t_1 to t_3 . In contrast, the red edge has a higher probability to be abnormal because the red nodes before t_3 are distant and have no structural connection.

financial systems [5], [6], IoT networks [7], and infrastructure facility networks [8], etc. An essential task in these dynamic graph scenarios is anomaly detection [9], which aims to identify the deviated patterns such as sudden changes in subgraph structure or node attributes [10], [11], [12]. For example, rumors in social networks may cause people to panic and run on public resources in a short period of time, and the spread of rumors can be regarded as an anomaly. Accurately detecting these anomalies can help prevent such panic and save massive public resources, which has become a research hot spot recently [13], [14], [15], [16], [17].

Though essential and eager to be solved, recent studies for anomaly detection in dynamic graphs mainly fall short in two aspects. **The first challenge lies in dynamic graph representation learning, where informative raw features are unavailable and the topological structures and the temporal dynamics have been modeled separately without considering the mutual relationship between the two modalities.** From the data perspective, due to the explosive of graph data scale and rising labor costs, it is expensive and infeasible to construct large amounts of informative features in real-world graph datasets. From the method perspective, previous dynamic graph methods [14], [15] mostly combine the static graph representation learning methods (e.g. GCN) with recurrence networks (e.g. GRU) by simply feeding the outputs of the former to the latter, while deeper connections between the structural and temporal information may left unexplored. Fig. 1 gives a simple demonstration explaining why both information should be considered jointly. The green line in t_3 appears to be a normal edge because the structural connection between the green nodes is getting stronger over time, while the red line is likely to be abnormal

Manuscript received 22 December 2022; revised 12 October 2023; accepted 26 October 2023. Date of publication 30 October 2023; date of current version 10 June 2024. This work was supported in part by the Zhejiang NSF under Grant LR21F020004, in part by NSFC under Grant 62272411, in part by the National Key Research and Development Program of China under Grant 2018AAA0101900, and in part by the Ant Group and Alibaba-Zhejiang University Joint Research Institute of Frontier Technologies. Recommended for acceptance by Muhammad Aamir Cheema. (Corresponding Author: Juncheng Li.)

Jianhao Guo, Siliang Tang, Juncheng Li, and Kaihang Pan are with the Department of Computer Science, Zhejiang University, Hangzhou 310027, China (e-mail: guojianhao@zju.edu.cn; siliang@zju.edu.cn; junchengli@zju.edu.cn; pankaihang@zju.edu.cn).

Lingfei Wu is with the Content and Knowledge Graph Group, Pinterest, New York, NY 10018 USA (e-mail: teddy.lfwu@gmail.com).

Digital Object Identifier 10.1109/TKDE.2023.3328645

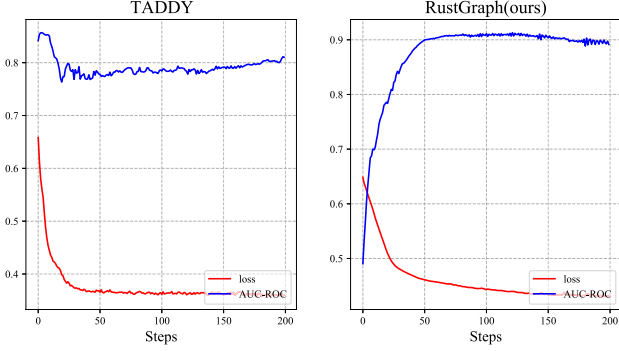


Fig. 2. Training curves between TADYY [16] and our RustGraph under 10% injected label noise in UCI Message dataset. Note that the value of loss curve only reflects the relative trend, and its value does not correspond exactly to the y-axis.

as the red nodes are distant and have no structural connection before t_3 . Simply combining two individual networks may fail to capture such patterns and result in performance degradation.

Second, most dynamic graph anomaly detection models have been trained by unreliable noisy labels and are vulnerable to subtle perturbations. Since there are few anomalies in real-world scenarios for anomaly detection in dynamic graphs, and high-quality labeled data are hard and expensive to collect, existing methods utilize negative sampling to generate pseudo anomalies and use these pseudo labels to train the model. For example, AddGraph [14] and StrGNN [15] generate anomalous edges by randomly replacing the endpoints of a given normal edge, and TADYY [16] randomly selects edges that do not appear at the current time as pseudo anomalies. However, such random negative sampling strategies may introduce the problem of noisy labels, where normal edges could be wrongly assigned to the abnormal class, or vice versa. Fig. 2 compares training curves of different models under 10% label noise, where TADYY is the baseline of previous work, and RustGraph is our proposed model. We can see that, as the training proceeds, the loss of TADYY decreases, but the AUC-ROC value does not increase, which shows that the model has overfitted the noise data. This example validates that the existence of noisy labels could largely deteriorate the model performance, which is more severe in the anomaly detection task, where human annotation is unavailable. Despite the significance of model robustness to noisy labels in anomaly detection, it has been rarely discussed in previous works.

To overcome these two obstacles, we propose RustGraph, a **RobUst** framework with **Structural-Temporal** dependency for anomaly detection in dynamic **GRAPHS**. **To solve the first limitation**, we design a structural-temporal graph encoding network, including a designed auto-encoder VST-GAE that simultaneously encodes both structural and temporal information with an informative prior. On top of that, we introduce a temporal consistent dynamic graph contrastive learning module named TC-GCL that encourages the model to capture fine-grained temporal dependencies. **As for the second limitation**, we are the first to formulate the problem of noisy labels brought by negative sampling in dynamic graph anomaly detection tasks.

To solve the problem, we design a robust graph anomaly detector that utilizes graph structural information for better model robustness and data reliability. For model robustness, we use the reconstructed structural information from graph embeddings to design a regularization term. For data reliability, we leverage the above structural information to iteratively update graph labels to get cleaner data.

In summary, we use the structural information learned in the graph encoding part to help the model fight against noisy labels. In turn, solving the noise problem can better help the learning of graph embeddings, thus forming a closed loop of mutual gain. The major contributions of our work can be summarized as follows:

- We propose RustGraph, a robust self-supervised framework that jointly learns both structural and temporal knowledge for anomalous edge detection in dynamic graphs.
- We design a variational structural-temporal graph auto-encoder, denoted as VST-GAE, which sufficiently combines the structural and temporal attributes in dynamic graphs to learn graph representations for anomaly detection. Besides, we propose TC-GCL, a temporal consistent dynamic graph contrastive learning module that fully exploits fine-grained temporal reliance between snapshots. With above two modules combined, we obtain a powerful dynamic graph encoder for better representations.
- To our best knowledge, we are the first to formulate the problem of noisy labels in dynamic graph anomaly detection, and we present a robust anomaly detector that utilizes graph structural information as strong regularization and data cleansing signal.
- Extensive experiments on six real-world datasets reach state-of-the-art performance with an average 3.64% performance gain and validate the effectiveness of our framework.

II. RELATED WORKS

A. Anomaly Detection in Dynamic Graphs

Existing dynamic graph representation learning methods can be divided into two categories [18], discrete-time dynamic graphs (DTDG) and continuous-time dynamic graphs (CTDG). DTDGs are represented as a sequence of snapshots taken at different time intervals, denoted as $\mathbb{G} = \{\mathcal{G}^1, \dots, \mathcal{G}^T\}$. On the other hand, CTDGs are characterized as a collection of events with associated timestamps. Each event is represented as (u_i, v_i, t_i) , where u_i, v_i are the endpoints of the event and t_i is the timestamp. In this work, we specifically focus on the setting of DTDGs. This choice is motivated by the fact that there are limited studies addressing the CTDG setting in the context of dynamic graph anomaly detection. Furthermore, many state-of-the-art techniques and publicly available datasets for this task are based on the DTDG setting. However, we acknowledge the potential importance of CTDGs and the possibility of exploring them in future research endeavors.

As an essential task in dynamic graphs, anomaly detection has aroused extensive research interest recently [9], [19]. Early methods focused on traditional machine learning methods.

GOutlier [20] uses structural connectivity to score the outliers. CM-Sketch [21] utilizes empirical anomaly metrics to detect anomalous edges.

Recently, deep learning methods have become mainstream for dynamic graph anomaly detection tasks and have achieved ideal performance compared to the above traditional learning methods. NetWalk [13] uses a random walk-based auto-encoder to learn low-dimensional representations, then clusters these node representations to detect anomalies. AddGraph [14] detects anomalous edges end-to-end. It encodes graphs with GCN [22] and aggregates short-term and long-term encoding with an attention mechanism through a GRU network. StrGNN [15] constructs h-hop enclosing subgraph stream of each edge and leverages stacked GCN and GRU to capture structural and temporal information. TADDY [16] uses substructure sampling to get graph embedding and uses transformer as the encoder.

However, as we mentioned before, these methods mainly fall short in two aspects. First, they encode the structural and temporal information separately, which may neglect the inter-dependencies between the two perspectives. And the node representation learning methods could be further improved for better anomaly detection results. Second, the problem of noisy labels is largely overlooked. Given the fact that there do not exist ground-truth anomalies in these datasets, the negative sampling strategies they adopt may introduce a massive number of low-quality labels. The work of this paper attempts to bridge this gap in these aspects.

B. Graph Representation Learning

1) *Graph Auto-Encoders*: Auto-encoders (AEs) are neural network architecture that maps the original information into a low-dimensional hidden feature space through the encoder, then uses the decoder to reconstruct the original information [23]. It can be used as a feature extractor for representation learning. Graph auto-encoder [24] extends the idea of AE to graph neural networks [22], [25] that encodes the graph structural information and attribute features of nodes. However, traditional AEs only learn features based on the reconstruction error and lack the ability to further regularize the learned feature distribution, limiting their generalization ability. In contrast, the variational graph auto-encoder (VGAE [24]) learns the probability distribution of the feature matrix, leading to improved performance in various graph-related tasks. VGRNN [26] further extends VGAE to dynamic graphs, which uses the information of previous timestamps to help approximate the current representation. However, VGRNN approximates the prior distribution only with limited information, which lacks the task-specific inductive biases for anomaly detection. In this work, we propose a dynamic variational graph auto-encoder that utilizes the eigenvectors of the graph Laplacian [27] as a more informative prior specifically for graph anomaly detection. We implement this with a Graph-GRU structure, which will be discussed in detail in Section IV-A1.

2) *Graph Contrastive Learning*: In recent years, graph contrastive learning, which does not require large amounts of labeled data and learns by maximizing mutual information of related

views [28], [29], has become a mainstream method for graph representation learning. DGI [30] contrasts between patch representations and corresponding high-level summaries of graphs. GraphCL [31] studies various graph augmentations for generating different views. GRACE [32] focuses on contrasting different views at the node level. GCCAD [33] makes attempts to leverage contrastive learning for graph anomaly detection, while they mainly focus on the static graph.

Recently, more research has begun to focus on the field of dynamic graphs. DDGCL [34] proposes a debiased GAN-type loss by contrasting two nearby temporal views of the same node identity. TCL [35] replaces the GNN encoder with a co-attention Transformer to model inter-dependencies of the contrasting views.

To model fine-grained temporal dependencies, we propose a simple yet effective dynamic graph contrasting method based on the temporal consistency between consecutive snapshots, which will be elaborated in Section IV-A2.

C. Learning With Noisy Labels on Graphs

Noisy label learning aims to solve the problem of how a learning algorithm can cope with incorrect training examples. While there is a significant body of work on the problems of noisy labels, most of them focused on the computer vision area, which ranges from estimating noise transition matrix [36], [37], regulating objective function [38], [39], [40], [41], [42], reweighting [43], [44] or selecting clean samples [45], [46], and so on. Compared to the prosperity of noisy label learning in CV, only a few works discussed the problem of training robust GNNs against label noise. This is mainly because graphs are non-IID data, and noise could propagate between nodes via the message passing mechanism of GNNs [47], which adds more difficulty to this problem. In the limited literature, [48] constructs a framework that identifies missing links and noisy links by leveraging mutual influence of them. [49] uses the clustering labels as regularization to punish overfitting to noisy labels. UnionNET [50] performs sample reweighting according to the class probability distributions. NRGNN [47] proposes to link the unlabeled nodes with labeled ones and generate pseudo labels to bring more clean label information. Though effective to some extent, these methods either lack graph-specific information guidance or introduce extra computational overhead to correct the label noise.

Different from above methods, in Section IV-B we utilize the structural representation learned in the VST-GAE encoder, as the constraint of robust regularization and signal for smooth label correction, yielding remarkable results with minuscule overheads.

III. PROBLEM FORMULATION

This section gives the necessary preliminaries of anomalous edge detection in dynamic graphs.

Given a discrete-time dynamic graph $\mathbb{G} = \{\mathcal{G}^1, \dots, \mathcal{G}^T\}$ with a maximum timestamp of T , where $\mathcal{G}^t = (\mathcal{V}^t, \mathcal{E}^t)$ represents a graph snapshot at timestamp t . $\mathcal{V}^t$ and $\mathcal{E}^t$ are the node set and edge set at timestamp t , respectively. We use $(\mathbf{A}^t, \mathbf{X}^t)$ to represent the topological and attributive information of graph $\mathcal{G}^t$.

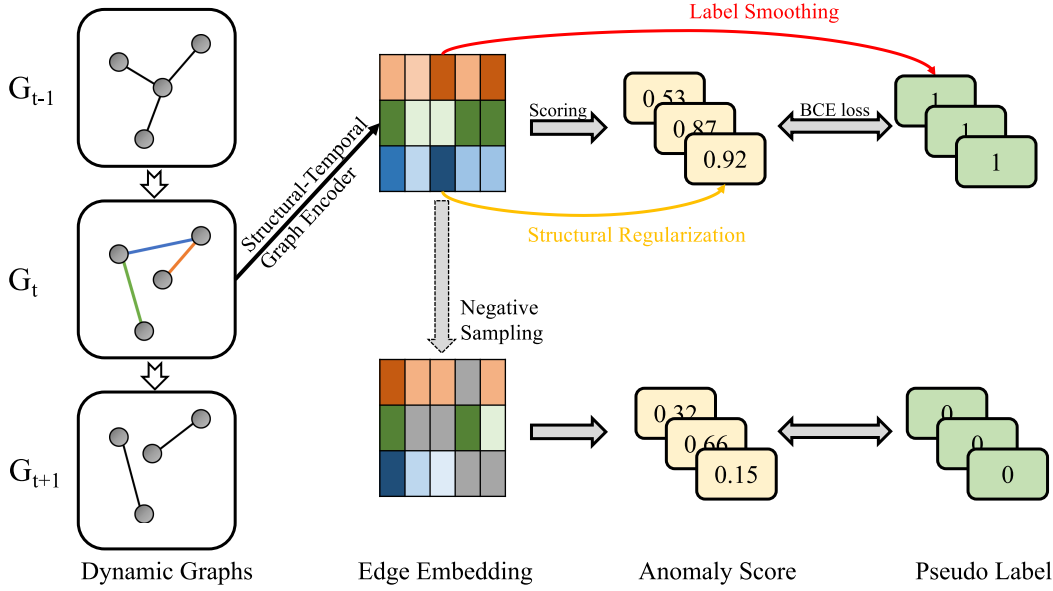


Fig. 3. Framework of RustGraph. The input dynamic graph series are encoded by our structural-temporal encoder (including VST-GAE and TC-GCL), then the learned graph embeddings are used to score the edge anomaly probability. Beside that, we use structural regularization and label smoothing to prevent the model from overfitting the noisy labels introduced by negative sampling.

$\mathbf{A}^t \in \mathbb{R}^{N_t \times N_t}$ is the adjacency matrix, where $\mathbf{A}_{ij}^t = 1$ if there exists an edge e_{ij}^t between node v_i^t and v_j^t , otherwise 0, and $\mathbf{X}^t \in \mathbb{R}^{N_t \times d}$ is the node attribute matrix, where $N_t = |\mathcal{V}^t|$ and d is the feature dimension of nodes. Note that nodes and edges can appear or disappear as time changes, the size and content of $(\mathbf{A}^t, \mathbf{X}^t)$ also vary.

In this paper, we design an anomaly detection algorithm $f(\cdot) \rightarrow [0, 1]$ on the edges e_{ij}^t of the dynamic graph that can detect anomalous events from both the structural and temporal perspectives.

Our model is trained in a self-supervised fashion following previous works [14], [15], [16]. Specifically, in the training phase, the data are unlabeled and assumed to be all normal, which is a natural phenomenon in real-world scenarios (In the real world, most of the time the data are normal and anomalies occur occasionally). We use these unlabeled data to train the model. In the testing phase, we inject anomalous edges [13], where each edge $e_{ij}^t = (v_i^t, v_j^t) \in \mathcal{E}^t$ is associated with a label $y_{ij}^t \in \{0, 1\}$, with 0 corresponds to abnormal edges and 1 corresponds to normal edges, respectively. Then we evaluate the performance of the trained model $f(e_{ij}^t)$.

IV. METHOD

In this section, we will introduce the design of RustGraph from two perspectives. First is the structural-temporal graph encoder that consists of VST-GAE and TC-GCL module. Through the encoder, the model will be able to learn the coupled structural-temporal graph representations from the perspectives of generative and contrastive learning; Second is the graph anomaly detector, which is noise-robust owing to the design of structure regularization and label smoothing. Fig. 3 gives an overview of the framework.

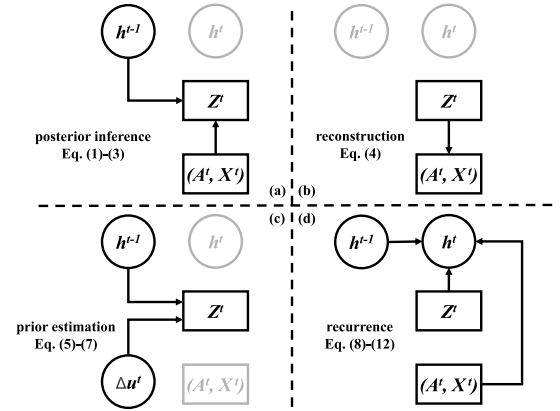


Fig. 4. Process of VST-GAE. (a) Posterior inference as in (1)–(3); (b) reconstruction as in (4); (c) prior estimation using informative prior as in (5)–(7); (d) the Graph-GRU recurrence as in (8)–(12).

A. Structural-Temporal Graph Encoder

1) VST-GAE. *Variational Structural-Temporal Graph Auto-Encoder*: In order to learn graph representations that capture the coupled information from structural and temporal perspectives, we propose a **Variational Structural-Temporal Graph Auto-Encoder**, namely VST-GAE, by jointly reasoning over both graph structural and temporal dependency information into one unified framework for dynamic graph representation learning. Fig. 4 gives a figurative illustration of the process in VST-GAE.

Initially, we focus on the static case, which applies variational auto-encoders to individual graph snapshots. The latent variables, denoted as $\mathbf{Z}$, are sampled from a probability distribution that is regularized by the graph encoder. Subsequently, the latent variables $\mathbf{Z}$ are used to reconstruct the original input.

For dynamic graphs, we can simply extend this approach by incorporating a recursive network to capture temporal evolution. However, this strategy overlooks the inherent relationship between the structural and temporal aspects, as the recurrence network lacks explicit knowledge of the graph structure.

In VST-GAE, we address this limitation by integrating structural and temporal information in a more granular and explicit manner. During the inference process, we introduce the hidden state vector $\mathbf{h}^{t-1}$, which represents the historical information from previous timestamps ($t-1$), and incorporate it into the GNN encoder. This integration allows us to effectively leverage both the structural and temporal characteristics of the dynamic graph. Formally, for a given graph snapshot $\mathcal{G}^t = (\mathbf{A}^t, \mathbf{X}^t)$, we infer the posterior distribution of the node embedding $\mathbf{Z}^t$ using the following equation:

$$q(\mathbf{Z}^t | \mathbf{A}^t, \mathbf{X}^t, \mathbf{h}^{t-1}) = \prod_{i=1}^{N_t} q(\mathbf{Z}_i^t | \mathbf{A}^t, \mathbf{X}^t, \mathbf{h}^{t-1}) \quad (1)$$

$$\mathbf{Z}_i^t \sim \mathcal{N}(\boldsymbol{\mu}_{i,enc}^t, \text{diag}((\boldsymbol{\sigma}_{i,enc}^t)^2)) \quad (2)$$

$$(\boldsymbol{\mu}_{enc}^t, \boldsymbol{\sigma}_{enc}^t) = \text{GNN}(\mathbf{A}^t, \mathbf{X}^t \oplus \mathbf{h}^{t-1}) \quad (3)$$

where $q()$ denotes posterior probability distribution, $\mathcal{N}()$ represents Gaussian distribution, $\boldsymbol{\mu}_{enc}^t, \boldsymbol{\sigma}_{enc}^t$ are the matrices of means and standard deviations, respectively. The encoder GNN can be any message passing networks (we use GraphSAGE [25] here), and $\oplus$ is the concatenation operator. The node embedding $\mathbf{Z}^t \in \mathbb{R}^{N_t \times d}$ is inferred by Gaussian distribution using the reparameterization trick [51].

With the inferred latent variables $\mathbf{Z}^t$, the original $\mathbf{A}^t$ and $\mathbf{X}^t$ are reconstructed by feeding $\mathbf{Z}^t$ to a non-linear decoder. Here we employ MLP as a decoder to reconstruct $\mathbf{X}^t$, and a simple inner-product decoder to reconstruct $\mathbf{A}^t$:

$$\hat{\mathbf{X}}^t = \text{MLP}(\mathbf{Z}^t), \hat{\mathbf{A}}^t = \sigma(\mathbf{Z}^t \mathbf{Z}^{t\top}) \quad (4)$$

where σ is the sigmoid function.

As for the prior, instead of imposing a standard Gaussian $p(\mathbf{Z}) = \prod_i \mathcal{N}(\mathbf{z}_i | 0, \mathbf{I})$ to estimate prior like VGAE [24] did, we use history information, that is, the features of previous steps to model the prior distribution. First, in a time series, the state at the current moment is affected by the previous timestamps, so we introduce $\mathbf{h}$ as auxiliary information. Second, changes in graph structure at adjacent timestamps are particularly significant in the dynamic graph anomaly detection task, because sudden and abrupt changes in graph structure are likely to herald the occurrence of anomalies. We therefore hope to amplify this information by directly introducing graph structure changes at adjacent timestamps. The eigenvectors of graph Laplacian have been proven to be an effective signal of graph structure and geometric information [27]. In this regard, we calculate the difference of the eigenvectors of the graph Laplacian between current and previous steps, denoted as $\Delta \mathbf{u}^t = |\mathbf{u}^t - \mathbf{u}^{t-1}|$, and use it as the parameters of the prior distribution. In detail, the prior distribution is estimated by:

$$p(\mathbf{Z}^t | \mathbf{h}^{t-1}, \Delta \mathbf{u}^t) = \prod_{i=1}^{N_t} p(\mathbf{Z}_i^t | \mathbf{h}^{t-1}, \Delta \mathbf{u}^t) \quad (5)$$

$$\mathbf{Z}_i^t \sim \mathcal{N}(\boldsymbol{\mu}_{i,prior}^t, \text{diag}((\boldsymbol{\sigma}_{i,prior}^t)^2)) \quad (6)$$

$$(\boldsymbol{\mu}_{prior}^t, \boldsymbol{\sigma}_{prior}^t) = \text{MLP}(\mathbf{h}^{t-1} \oplus \Delta \mathbf{u}^t) \quad (7)$$

where $p()$ is the prior probability distribution, $\boldsymbol{\mu}_{prior}^t, \boldsymbol{\sigma}_{prior}^t$ are the prior means and standard deviations matrices.

Consider the hidden state vector $\mathbf{h}^t$, it is a function of previous graph signals, including $\mathbf{A}^{\leq t}, \mathbf{X}^{\leq t}$, and $\mathbf{Z}^{\leq t}$. Using a simple recurrence function like LSTM or GRU to update it may neglect the graph topological information. Therefore, we replace the fully connected layers in the GRU with GNN encoders, namely GraphGRU, so that the recurrence part can process graph data. Specifically, the hidden state $\mathbf{h}^t$ is updated by:

$$\mathbf{X}'^t = \phi^x(\mathbf{X}^t) \oplus \phi^z(\mathbf{Z}^t) \quad (8)$$

$$\mathbf{z}^t = \sigma(\text{GNN}_{xz}(\mathbf{A}^t, \mathbf{X}'^t) + \text{GNN}_{hz}(\mathbf{A}^t, \mathbf{h}^{t-1})) \quad (9)$$

$$\mathbf{r}^t = \sigma(\text{GNN}_{xr}(\mathbf{A}^t, \mathbf{X}'^t) + \text{GNN}_{hr}(\mathbf{A}^t, \mathbf{h}^{t-1})) \quad (10)$$

$$\mathbf{h}' = \tanh(\text{GNN}_{xh}(\mathbf{A}^t, \mathbf{X}'^t) + \text{GNN}_{hh}(\mathbf{A}^t, \mathbf{h}^{t-1} \circ \mathbf{r}^t)) \quad (11)$$

$$\mathbf{h}^t = (\mathbf{1} - \mathbf{z}^t) \circ \mathbf{h}^{t-1} + \mathbf{z}^t \circ \mathbf{h}' \quad (12)$$

where ϕ^x and ϕ^z are project functions such as MLP that project $\mathbf{X}^t$ and $\mathbf{Z}^t$ into the similar hidden space, and $\circ$ is the Hadamard product.

The overall loss function for VST-GAE module is derived by the variational lower bound:

$$\begin{aligned} \mathcal{L}_{\text{gen}} = & \mathbb{E}_{q(\mathbf{Z}^t | \mathbf{A}^t, \mathbf{X}^t, \mathbf{h}^{t-1})} \log p(\mathbf{A}^t, \mathbf{X}^t | \mathbf{Z}^t) \\ & - \text{KL}(q(\mathbf{Z}^t | \mathbf{A}^t, \mathbf{X}^t, \mathbf{h}^{t-1}) \| p(\mathbf{Z}^t | \mathbf{h}^{t-1}, \Delta \mathbf{u}^t)) \end{aligned} \quad (13)$$

where the first term is the reconstruction loss, and the second term is the Kullback-Leibler divergence between the posterior and prior distributions of $\mathbf{Z}^t$, which is used as a regularization term that secures the probability distribution of $\mathbf{Z}^t$ close to the informative prior.

2) **TC-GCL. Temporal Consistent Graph Contrastive Learning:** Although the design of hidden state $\mathbf{h}$ in VST-GAE implicitly passes temporal information through the recurrence part, such design only considers the sequence dependency of previous snapshots, which may neglect more complicated reliance between neighboring snapshots from both directions. To preserve semantic consistency between consecutive snapshots, we propose an augmentation-free graph contrasting objective, namely **Temporal Consistent Graph Contrastive Learning (TC-GCL)**, which treats neighboring snapshots of an anchor timestamp t as augmented views, and maximizes the mutual information of the same node on different views.

Specifically, after we obtain the embedding matrix $\mathbf{Z}^t$ for a given timestamp t , the previous and next neighboring snapshots of t are sampled as augmented views $\mathcal{Z}^{aug(t)} = \{\mathbf{Z}^{t-1}, \mathbf{Z}^{t+1}\}$. For generality, we use $\mathbf{Z}^*$ for one augmented view hereinafter. It is worth noting that this method does not use any augmentations such as edge perturbation, but still keeps the consistency of

semantics between different views and reduces computational cost.

For any node i that appears on both $\mathbf{Z}^t$ and $\mathbf{Z}^*$, the node representations of the same node identity i are selected as positive pairs, i.e., $(\mathbf{Z}_i^t, \mathbf{Z}_i^*)$, and other nodes $(\mathbf{Z}_i^t, \mathbf{Z}_j^*)|_{i \neq j}$ are seen as negative pairs. Then a node-level contrastive loss is applied to distinguish the embeddings of the same node in these two views from the other node embeddings. We formalize the similarity metric as $s(\mathbf{u}, \mathbf{v}) = \text{cosine_similarity}(g(\mathbf{u}), g(\mathbf{v}))$, where g is a non-linear projection for general representations [52]. With above preliminaries, the augmentation-free contrasting objective at time t is written as:

$$\mathcal{L}_{\text{con}} = - \sum_{\mathbf{Z}^*} \sum_i^{\mathbb{Z}^{aug(t)} \mathbf{Z}^t \cap \mathbf{Z}^*} \log \left(\frac{e^{s(\mathbf{Z}_i^t, \mathbf{Z}_i^*)/\tau}}{e^{s(\mathbf{Z}_i^t, \mathbf{Z}_i^*)/\tau} + \sum_{k=1}^N \mathbb{1}_{[k \neq i]} e^{s(\mathbf{Z}_i^t, \mathbf{Z}_k^*)/\tau}} \right) \quad (14)$$

where τ is the temperature parameter, $\mathbb{1}$ is an indication function that equals to 1 iff $k \neq i$. Note that this formula only calculates the common nodes that appear on both views, which means our model can handle node adding or dropping, and edge adding or dropping scenarios.

Combining VST-GAE and TC-GCL, the overall objective function for dynamic graph representation learning is written as:

$$\mathcal{L}_{\text{enc}} = \mathcal{L}_{\text{gen}} + \mathcal{L}_{\text{con}} \quad (15)$$

B. Robust Anomaly Detector With Structure Regularization and Label Smoothing

From the above structural-temporal graph encoder we can obtain the features $\mathbf{Z}^t \in \mathbb{R}^{N_t \times d}$ at timestamp t , where each row of $\mathbf{Z}^t$ represents the feature of the corresponding node. For edge $\mathbf{e}_{ij}^t \in \mathcal{E}^t$, we use the mean value of its endpoints as the representation: $\mathbf{e}_{ij}^t = (\mathbf{Z}_i^t + \mathbf{Z}_j^t)/2$. Then we can compute its anomaly score by a learnable function:

$$f(\mathbf{e}_{ij}^t) = \sigma(\mathbf{e}_{ij}^t \mathbf{W} + b) \quad (16)$$

where σ is the sigmoid activation function, $\mathbf{W} \in \mathbb{R}^d$ and $b \in \mathbb{R}$ are learnable weight matrix and bias.

Since there do not exist any ground-truth anomalies in the training data, we cannot directly train the detector to distinguish between normal and abnormal edges yet. Previous works use negative sampling techniques to generate pseudo anomalous edges, such as randomly selecting non-existing edges [16] or randomly replacing each edge's endpoint [14]. With generated pseudo labels, the anomaly detection problem can be treated as a binary classification task using binary cross-entropy loss:

$$\mathcal{L}_{\text{bce}} = - \sum_{\mathbf{e}_{ij}^t \in \mathcal{E}^t} \{ (1 - y_{ij}) \log(1 - f(\mathbf{e}_{ij}^t)) + y_{ij} \log(f(\mathbf{e}_{ij}^t)) \} \quad (17)$$

where y_{ij} is the given label of the edge $\mathbf{e}_{ij}^t$.

However, even in the presence of ground-truth labels, the introduction of negative sampling to generate pseudo labels will

bring about the problem of noisy labels, not to mention in the current unsupervised and unlabeled scenario, which will largely reduce the confidence of the data and harm the robustness of the model.

To prevent model from overfitting corrupted labels, one solution is to exploit high-level constraint as regularization. Aside from label guidance, another high-level signal is graph structure, which provides huge amounts of information of the graph. Also, in the graph anomaly detection task, the change in structure connectivity may indicate potential anomalies (e.g. a sudden surge of connection from an inactive node could infer malicious activities). Based on such observations, we propose to utilize explicit structure information as regularization against the noises.

To simplify the network design and reduce computation cost, we reuse the embedding $\mathbf{Z}^t$ encoded by the VST-GAE module, which carries necessary information to reconstruct the graph: $\hat{\mathbf{A}}^t = \sigma(\mathbf{Z}^t \mathbf{Z}^{t\top})$, as the reiteration in (4). The reconstructed adjacency matrix $\hat{\mathbf{A}}^t$ reveals the existence probability between every nodes pair in the graph, a higher value at $\hat{\mathbf{A}}_{ij}^t$ means node i and j are more likely to be connected, and vice versa. On one hand, the regularization from $\hat{\mathbf{A}}^t$ can prevent the model from overfitting the noisy labels when the label is corrupted. Formally, for edge $\mathbf{e}_{ij}^t$, its anomaly score $f(\mathbf{e}_{ij}^t)$ is regulated by:

$$\mathcal{L}_{\text{reg}} = \sum_{\mathbf{e}_{ij}^t \in \mathcal{E}^t} \|\hat{\mathbf{A}}_{ij}^t - f(\mathbf{e}_{ij}^t)\| \quad (18)$$

On the other hand, $\hat{\mathbf{A}}^t$ can also be leveraged as additional signal for label correction to iteratively get more reliable labels, which is done by an exponential moving average process:

$$\mathbf{Y}^t = \epsilon \mathbf{Y}^t + (1 - \epsilon) \hat{\mathbf{A}}^t \quad (19)$$

where ϵ is decay rate in the range $[0, 1]$.

It is worth noting that the model learns through noisy data by utilizing the structural information from previous graph encoder, which in turn improves the quality of graph embeddings with more reliable data. In such a loop, the structural-temporal graph encoder and the robust anomaly detector benefit from each other, improving the overall performance on dynamic graph anomaly detection task.

C. Training Procedure

Combining the structural-temporal information mining objective, the anomaly detection objective, and the noise-resistant regularization objective, the final training objective can be formalized as:

$$\mathcal{L} = \alpha \mathcal{L}_{\text{enc}} + \beta \mathcal{L}_{\text{bce}} + \gamma \mathcal{L}_{\text{reg}} \quad (20)$$

where α , β , and γ are hyperparameters that control the volume of each component loss. Without bells and whistles, we set all of them as 1 by default, and more experiments discussing the influence of these hyperparameters are in Section V-E3. We also summarize the overall training procedure in Algorithm 1.

Algorithm 1: Algorithm of RustGraph.

Input: Dynamic graph $\mathbb{G} = \{\mathcal{G}^1, \dots, \mathcal{G}^T\}$, Edge anomaly labels $\mathbb{Y} = \{\mathbf{Y}^1, \dots, \mathbf{Y}^T\}$, training epochs M .

Output: Edge anomaly probabilities at each timestamp.

```

1 Initialize network parameters;
2 for  $i = 1$  to  $M$  do
3   for  $t = 1$  to  $T$  do
4     Calculate  $\mathcal{L}_{\text{enc}}$  and infer graph embedding  $\mathbf{Z}^t$ 
       using Equation (1) - (15)
5     Calculate edge anomaly score  $f(\mathbf{e}^t)$  using
       Equation (16)
6     Calculate  $\mathcal{L}_{\text{bce}}$  using Equation (17)
7     Calculate  $\mathcal{L}_{\text{reg}}$  using Equation (18)
8     Update  $\mathbf{Y}^t$  using Equation (19)
9   end
10  Accumulate each timestamp's loss and get the
    final loss function using Equation (20)
11  Back propagate gradients and update model
    parameters
12 end

```

D. Complexity Analysis

In this subsection we briefly analyze the time complexity of the proposed RustGraph model. Note for simplicity, we assume the input and output dimension of the network are both d . This analysis is based on a dynamic graph series of length T , with an average number of nodes n and edge e at each timestamp.

For a L -layer GraphSAGE model which samples uniformly a fixed number k of 1-hop neighbors, the time complexity is $\mathcal{O}(Ld^2 k^L)$. For the MLP used in our method, we assume it has one layer and the time complexity is $\mathcal{O}(nd^2)$. For the GRU unit, we use the conclusion from [53], which is $\mathcal{O}(Td^2)$.

According to above analysis, for the VST-GAE module, the posterior inference and prior estimation bring the complexity of $\mathcal{O}(Ld^2 k^L + nd^2)$; the reconstruction process has $\mathcal{O}(nd^2 + n^2 d)$; and the recurrence part has $\mathcal{O}(Td^2)$. The overall complexity of VST-GAE is derived by adding these steps, which is $\mathcal{O}((2n + Lk^L + T)d^2 + n^2 d)$. We omit the constant and $Lk^L + T$ term due to their relatively small value, then the time complexity for VST-GAE is $\mathcal{O}(nd^2 + n^2 d)$. For TC-GCL module, we assume each node at timestamp t has n contrasting pairs, then the overall complexity for TC-GCL is $\mathcal{O}(n^2 d)$. For anomaly detector, the time complexity is $\mathcal{O}(e)$.

To sum up, the final time complexity of RustGraph is $\mathcal{O}(n^2 dT + nd^2 T + eT)$.

V. EXPERIMENTS

In this section, we conduct extensive experiments to validate the effectiveness of the proposed RustGraph framework. We will first introduce the dataset used in the experiments, along with the compared baseline methods and other necessary setups. Then we will demonstrate the empirical results of RustGraph, and we also conduct ablation studies to prove the effectiveness of the model design. After that, we will discuss the model performance

TABLE I
STATISTICS OF SIX PUBLIC DYNAMIC GRAPH DATASETS

Dataset	#Nodes	#Edges	Avg. Degree
UCI Messages	1,899	13,838	14.57
Digg	30,360	85,155	5.61
Email-DNC	1,866	39,264	42.08
Bitcoin-Alpha	3,777	24,173	12.80
Bitcoin-OTC	5,881	35,588	12.10
AS-Topology	34,761	171,420	9.86

under noisy labels. Finally, there are some discussions about hyperparameters sensitivity and graph properties.

A. Experiment Setup

1) *Datasets*: We evaluate the performance of RustGraph on six public dynamic graph datasets. Some statistics are summarized in Table I, and below are the descriptions of these datasets.

UCI Messages [54] is the directed network that contains sent messages between the users of an online community of students from the University of California, Irvine. A node represents a user. A directed edge represents a sent message. **Digg** [55] is the reply network of the social news website Digg. Each node in the network represents a website user, and each directed edge denotes that a user replied to another user. **Email-DNC** [56] is the directed network of emails in the 2016 Democratic National Committee email leak. Nodes in the network correspond to persons in the dataset. A directed edge in the dataset denotes that someone has sent an email to another person. **Bitcoin-Alpha** [57] is a who-trusts-whom network of people who trade using Bitcoin on a platform called Bitcoin Alpha. A node in the network denotes a user from the platform, and an edge means a user has rated another user. **Bitcoin-OTC** [58] is similar to Bitcoin-Alpha, which is another who-trusts-whom network of users on the platform Bitcoin OTC. **AS-Topology** [59] is the network connections between some autonomous systems of the Internet, where nodes represent autonomous systems and edges represent their connections.

2) *Baselines*: We compare RustGraph with nine state-of-the-art baseline methods described below.

Sedanspot [60] detects anomalies by using a holistic random walk-based scoring function to compare the connectivity changes. **CM-Sketch** [21] utilizes Count-Min sketch for approximating empirical anomaly metrics to detect anomalous edges. **node2vec** [61] combines depth-first traversal and breadth-first traversal in random walk subgraph sampling and uses Skip-gram for embedding learning. **Spectral Clustering** [62] learns embedding by maximizing the similarity between nodes in the neighborhood. **DeepWalk** [63] generates random walk-based subgraphs and also uses Skip-gram to learn node embedding. **NetWalk** [13] learns node embedding by random walking and uses dynamic clustering of the embeddings to detect anomalies. **AddGraph** [14] leverages extended temporal GCN with an attentional GRU to combine structural-temporal information, and proposes a Bernoulli distribution-based negative sampling strategy. **StrGNN** [15] extracts h -hop enclosing subgraphs to generate embeddings for the incoming structural-temporal model. **TADDY** [16] utilizes a substructure sampling strategy

TABLE II
ANOMALY DETECTION RESULTS ARE REPORTED IN THE AUC-ROC SCORE

Methods	UCI Messages			Digg			Email-DNC		
	1%	5%	10%	1%	5%	10%	1%	5%	10%
SedanSpot	0.7342	0.7156	0.7061	0.6976	0.6784	0.6396	0.7427	0.7362	0.7235
CM-Sketch	0.7320	0.6968	0.6835	0.6884	0.6675	0.6358	0.7053	0.6946	0.6876
node2vec	0.7371	0.7433	0.6960	0.7364	0.7081	0.6508	0.7391	0.7284	0.7103
Spectral Clustering	0.6324	0.6104	0.5794	0.5949	0.5823	0.5591	0.8096	0.7857	0.7759
DeepWalk	0.7514	0.7391	0.6979	0.7080	0.6881	0.6396	0.7481	0.7303	0.7197
NetWalk	0.7758	0.7647	0.7226	0.7563	0.7176	0.6837	0.8105	0.8371	0.8305
AddGraph	0.8083	0.8090	0.7688	0.8341	0.8470	0.8369	0.8393	0.8627	0.8773
StrGNN	0.8179	0.8252	0.7959	0.8162	0.8254	0.8272	0.8775	0.9103	0.9080
TADDY	0.8912	0.8398	0.8370	0.8617	0.8545	0.8440	0.9348	0.9257	0.9210
RustGraph	0.9128	0.9117	0.9124	0.8795	0.8577	0.8624	0.9855	0.9837	0.9806
Methods	Bitcoin-Alpha			Bitcoin-OTC			AS-Topology		
	1%	5%	10%	1%	5%	10%	1%	5%	10%
SedanSpot	0.7380	0.7264	0.7085	0.7346	0.7284	0.7156	0.6873	0.6742	0.6672
CM-Sketch	0.7146	0.7015	0.6887	0.7412	0.7338	0.7242	0.6687	0.6605	0.6558
node2vec	0.6910	0.6802	0.6785	0.6951	0.6883	0.6745	0.6821	0.6752	0.6668
Spectral Clustering	0.7401	0.7275	0.7167	0.7624	0.7376	0.7047	0.6685	0.6563	0.6498
DeepWalk	0.6985	0.6874	0.6793	0.7423	0.7356	0.7287	0.6844	0.6793	0.6682
NetWalk	0.8385	0.8357	0.8350	0.7785	0.7694	0.7534	0.8018	0.8066	0.8058
AddGraph	0.8665	0.8403	0.8498	0.8352	0.8455	0.8592	0.8080	0.8004	0.7926
StrGNN	0.8574	0.8667	0.8627	0.9012	0.8775	0.8836	0.8553	0.8352	0.8271
TADDY	0.9451	0.9341	0.9423	<u>0.9455</u>	<u>0.9340</u>	<u>0.9425</u>	<u>0.8953</u>	<u>0.8952</u>	<u>0.8934</u>
RustGraph	<u>0.9447</u>	0.9348	<u>0.9207</u>	0.9608	0.9578	0.9603	0.9746	0.9768	0.9761

The best result of each column is marked in bold font, and the second best is marked by underline.

and complex node embedding methods to feed the transformer model.

For the graph embedding methods [21], [60], [61], [62], [63], the K-means clustering-based detector [13] is used for anomaly detection. Other deep learning methods are trained end-to-end.

3) *Experiment Details*: Since there are no ground-truth anomalies in the original dataset, we follow the similar preprocess method in previous works [13], [14], [16] by injecting artificial anomalous edges. Node2vec [61] is used for generating raw features. The first 50% snapshots are used as the training set, and the rest are used as the testing set. For testing, we consider three different anomaly injection proportions (1%, 5%, 10%). We use GraphSAGE [25] as the GNN encoder in the Graph-GRU unit. To handle dynamic nodes addition/removal for ease of implementation, we enforce the shape $\mathbf{A}^t \in \mathbb{R}^{N \times N}$, $\mathbf{X}^t \in \mathbb{R}^{N \times d}$ of all snapshots to be same, where N is the total number of nodes that have appeared in the dynamic graph $\mathcal{G}$. And we maintain a mask vector $\mathbf{m}^t \in \mathbb{R}^N$ to indicate the occurrence of each node for current snapshot. This approach ensures that the input shapes for different snapshots remain consistent, allowing the encoder to handle node additions and removals appropriately.

4) *Metric*: In our experiment, we employ *Area Under the Curve of the Receiver Operating Characteristic* (AUC-ROC) as the performance metric, which ranges from 0 to 1, with a higher value indicating higher quality of the proposed method.

5) *Parameter Setting*: For the node embedding dimension d , we use 64 for AS-Topology, 512 for Email-DNC, and 128 for the rest. The snapshot sizes are set to be the same as [16] for a fair comparison, which is 1000 for UCI and BTC-OTC, 2000 for Email and BTC-Alpha, and 6000 for Digg and AS-Topology,

respectively. The layers of all GNN encoders used in the network are 2 by default.

During the training phase, we train UCI, BTC-OTC, Email for 200 epochs, and Digg, BTC-Alpha, AS-Topology for 400 epochs. The framework is optimized by Adam optimizer with the learning rate of 0.001 for UCI, Email, AS-Topology, and 0.0005 for Digg, BTC-Alpha, and BTC-OTC. The hyperparameters α, β, γ that control the loss component are set to 1 by default. The weight decay rate ϵ in (19) is set as 0.9.

B. Anomaly Detection Results

The results of dynamic graph edge anomaly detection on six datasets are shown in Table II, from which we can see:

- Out of 18 comparison items, we have 16 significantly better than and 2 on par with other baseline models, achieving state-of-the-art results. Compared with the best results of these baselines, our model achieves an average performance increase of **3.64%**. Especially on datasets AS-Topology, UCI Message, and Email-DNC, the average performance gain is **8.12%**, **5.63%**, and **5.61%**. The results prove that RustGraph can significantly improve the performance of dynamic graph anomaly detection under various settings.
- Our method achieves an average increase of **3.07%**, **3.99%**, **3.87%** when the anomaly ratio is 1%, 5%, 10%, respectively. Compared with other methods, the AUC values of RustGraph under different anomaly proportions have no obvious difference. It justifies that our method can maintain stability and robustness regardless of the settings.

TABLE III
ABLATION STUDY RESULTS FOR RUSTGRAPH

Methods	UCI Messages			Digg			Email-DNC		
	1%	5%	10%	1%	5%	10%	1%	5%	10%
RustGraph	0.9128	0.9117	0.9124	0.8795	0.8577	0.8624	0.9855	0.9837	0.9806
AE as encoder	0.5211	0.5129	0.5105	0.5248	0.5316	0.5291	0.5000	0.5000	0.5000
w/o informative prior	<u>0.9035</u>	<u>0.8865</u>	<u>0.8905</u>	<u>0.8749</u>	<u>0.8533</u>	<u>0.8552</u>	<u>0.9834</u>	<u>0.9816</u>	<u>0.9801</u>
w/o $\mathcal{L}_{\text{gen}}$	0.9010	<u>0.9046</u>	<u>0.9049</u>	0.8518	0.8348	0.8362	0.9744	0.9686	0.9641
w/o TC-GCL	0.8881	0.8916	0.8877	0.8491	0.8332	0.8376	0.9827	0.9808	0.9779

Methods	Bitcoin-Alpha			Bitcoin-OTC			AS-Topology		
	1%	5%	10%	1%	5%	10%	1%	5%	10%
RustGraph	0.9447	0.9348	0.9207	0.9608	0.9578	0.9603	0.9746	0.9768	0.9761
AE as encoder	0.5338	0.5135	0.5111	0.5392	0.5475	0.5460	0.8435	0.8312	0.8265
w/o informative prior	0.9275	0.9151	0.9060	<u>0.9600</u>	<u>0.9570</u>	<u>0.9583</u>	<u>0.9741</u>	0.9768	<u>0.9754</u>
w/o $\mathcal{L}_{\text{gen}}$	0.9305	0.9063	0.8914	0.9504	0.9503	0.9553	0.9457	0.9454	0.9443
w/o TC-GCL	<u>0.9435</u>	<u>0.9304</u>	<u>0.9184</u>	0.9474	0.9509	0.9532	0.9518	<u>0.9543</u>	0.9538

The best result of each column is marked in bold font, and the second best is marked by underline.

- The end-to-end deep learning methods have comprehensively surpassed the graph embedding methods. We assume this is because the deep learning methods simultaneously consider the structural and temporal information, and a bigger model also leads to better representation.

C. Ablation Study

To validate the effectiveness of each component in RustGraph, we conducted ablation studies by masking different components. Specifically, for the VST-GAE module, we first used the original auto-encoder (AE) instead of the variational auto-encoder (VAE) as the base model to study the necessity of the variational regularization. Then, we replaced the informative prior with a standard Gaussian distribution to assess the effectiveness of the informative prior. Finally, we masked the loss function $\mathcal{L}_{\text{gen}}$ while preserving the auto-encoder design. For the TC-GCL module, we removed the entire contrasting function $\mathcal{L}_{\text{con}}$. We will discuss the effect of the robust anomaly detection module in the next subsection. The other hyperparameters and training scheme were kept the same for a fair comparison. The results of ablation experiments are listed in Table III, and we can draw the conclusions:

- In all cases, removing either component leads to a notable performance drop. Specifically, using the auto-encoder as the base model results in invalid outcomes, where most values are around 50% AUC-ROC. Removing informative prior and $\mathcal{L}_{\text{gen}}$ leads to an average 0.74% and 1.85% drop in AUC-ROC, respectively. And the drop for removing TC-GCL is 1.45%. All the performance drops are large enough to influence the overall anomaly detection reliability. This observation indicates the necessity of each component in RustGraph.
- As an extreme case, using AE as an encoder results in completely unusable results. This may be because AE cannot provide regularization on the latent space and loses the ability to model both structural and temporal information.
- Different modules have different importance among datasets. For example, removing $\mathcal{L}_{\text{gen}}$ has less impact on the UCI dataset compared to removing TC-GCL, while on the

Email-DNC dataset we find the exact opposite effect. This finding supports the necessity of combining these modules, which are complementary in different scenarios.

- In most cases (except using AE as encoder), even the incomplete RustGraph network still outperforms other baseline models, which again demonstrates the performance advantage of our model on the dynamic graph anomaly detection task.

D. Impact of Noisy Labels

In this part we will discuss how the noisy labels affect the model performance and how the robust anomaly detector alleviate such influence. Given the fact that there is no open-source graph dataset with noisy labels, we use random label-flipping to manually inject noise to above datasets, and the noise ratio in the training set varies from 0 to 0.5, with a step size of 0.05. We compare the results of different methods to give a comprehensive view. For TADDY [16], we use their public codes to re-run the experiment. The results are shown in Fig. 5, and we can conclude that:

- RustGraph is robust to noisy labels under various settings: the blue curve remains stable as the noise ratio increases. In comparison, the performance of TADDY (the red curve) drops significantly with more noisy labels.
- It is the design of robust anomaly detector that empowers RustGraph the ability to counter noisy labels. According to the ablation study (the green curve), removing the structural regularization and label smoothing will lead to a notable performance drop, and as the noise ratio gets higher, the performance drop becomes more severe.
- When the noise ratio reaches 50%, the binary anomaly detection problem becomes intractable and the conventional methods have failed. But due to the existence of label smoothing design in our method (Section IV-B), the RustGraph model can obtain supervision signal other than the given labels. So even under such a extreme case, the performance can still maintain on a high level.

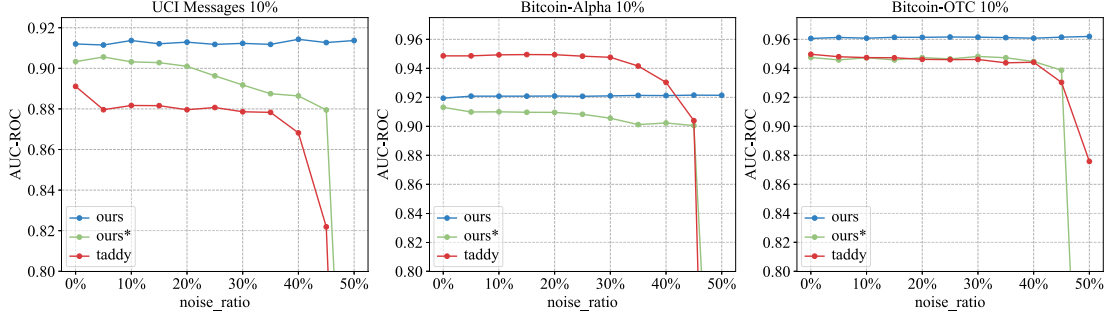


Fig. 5. AUC-ROC curves under different noise ratios. Because there is no ground-truth noisy labels, we use random label-flipping to inject noisy labels to original dataset. The **ours*** marker denotes the ablation study of RustGraph, where we remove the structural regularization and label smoothing. For results of TADDY, we use their open-source code to run the experiment.

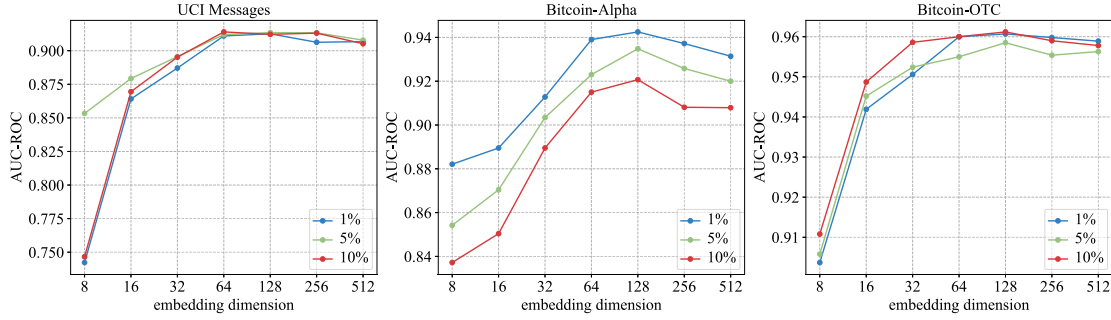


Fig. 6. Sensitivity analysis results for embedding dimensions d on three datasets with three kinds of anomaly ratio.

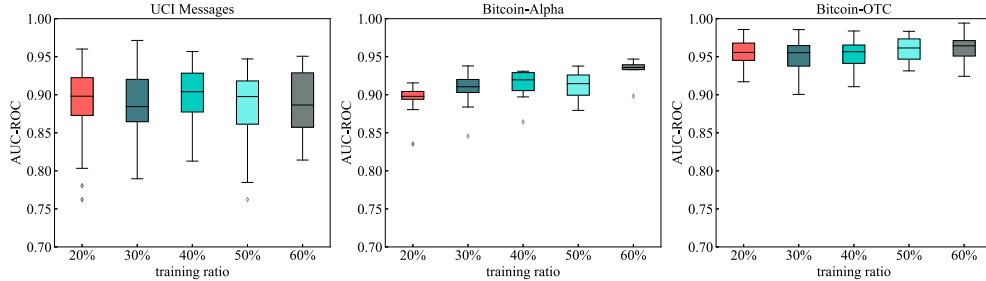


Fig. 7. Sensitivity analysis results for training ratios on three datasets in box plots. The diamond marks are outliers.

E. Sensitivity Analysis

We further analyze the influence of different hyperparameters on the experimental results, including the dimension of node embedding d , the training data ratio, the contribution of each component loss, and the snapshot sizes. The experiments are carried out on UCI Messages, Bitcoin-Alpha, and Bitcoin-OTC with 10% anomaly ratio and other parameters are kept the same.

1) *Embedding Dimensions*: In this experiment, we study the influence of embedding dimension d . The value of d ranges from $\{8, 16, 32, 64, 128, 256, 512\}$, and the results are shown in Fig. 6. As we can see, the AUC-ROC generally increases as the embedding dimension increases, and becomes optimal when d reaches 128. The AUC-ROC curves even slightly drop when d continually grows to 512. A possible explanation is that small d may not capture all the semantic information, thus reducing

the learning ability; while too large d makes the model hard to converge.

2) *Training Ratios*: To evaluate the effect of different training ratios, we conduct experiments with training ratios from $\{20\%, 30\%, 40\%, 50\%, 60\%\}$, and the results are summarized in Fig. 7. With the increase of training ratio, the AUC-ROC on Bitcoin-Alpha significantly increases, and other two datasets remain stable. We also discover that both variance and outliers decrease slightly as the training ratio increases, suggesting that sufficient data is beneficial for model training. Note that our method still achieves state-of-the-art results even for a relatively low training ratio (20%). This may be attributed to our structural-temporal graph encoder, which can capture rich information even with few training data.

3) *Contribution of Each Component Loss*: To further quantify the contribution of each component loss in (20), we set α, β , and γ in the range of $\{0.2, 0.4, 0.6, 0.8, 1.0\}$ on UCI Messages.

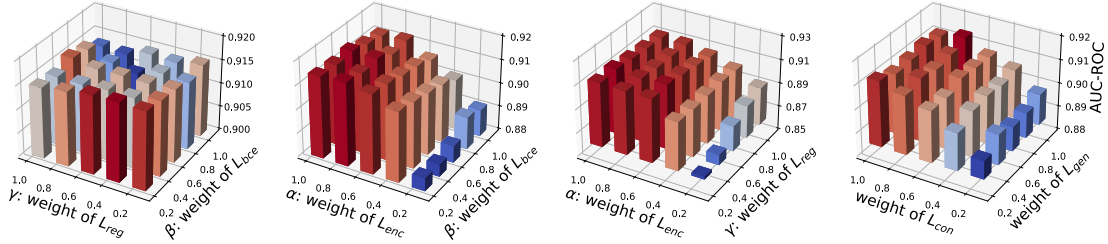


Fig. 8. Sensitivity analysis results for the volume of each component loss.

TABLE IV
SENSITIVITY ANALYSIS RESULTS FOR SNAPSHOT SIZES ON THREE DATASETS

Methods	UCI Messages					Bitcoin-Alpha					Bitcoin-OTC				
	100	500	1000	2000	5000	100	500	1000	2000	5000	100	500	1000	2000	5000
TADDY	0.6868	0.8434	0.8370	0.7968	0.7209	0.7842	0.9240	0.9427	0.9423	0.8994	0.7436	0.9207	0.9425	0.9414	0.9342
RustGraph	0.9053	0.9178	0.9124	0.9045	0.8714	0.9353	0.9412	0.9376	0.9207	0.8975	OOM	0.9637	0.9603	0.9479	0.9277

OOM indicates out of memory.

According to the first three subfigures in Fig. 8, the most sensitive hyperparameter is α , which controls the component of $\mathcal{L}_{enc}$. With the growth of α the detection performance also grows, this validates the importance of the structural-temporal graph encoder. On the other hand, the performance is rather stable as β and γ change. However, we can see that when the α is rather low, using larger β or γ could still improve the performance to some extent. From this experiment, we can conclude that our RustGraph model is relatively insensitive to the choice of hyperparameters, so we simply set all of them to 1 by default. Furthermore, we also study the roles of $\mathcal{L}_{gen}$ and $\mathcal{L}_{con}$ in (15) using the same range of hyperparameters, as shown in the last subfigure of Fig. 8. The performance increases with the increase of the weight of $\mathcal{L}_{gen}$ and $\mathcal{L}_{con}$, which demonstrates that the VST-GAE and TC-GCL module are helpful for anomaly detection tasks.

4) *Snapshot Sizes*: We also investigate the impact of varying snapshot sizes on the performance of anomaly detection, and the results are summarized in Table IV. Overall, the performance of both methods tends to increase and then decrease as snapshot sizes grow. We speculate that when snapshot sizes are too small, the number of nodes within an individual graph snapshot is limited, leading to insufficient structural information. On the other hand, overly large snapshot sizes result in fewer generated graph snapshots, leading to inadequate temporal information. These two extreme cases highlight the importance of making a reasonable choice for the snapshot size, as it can significantly impact the model's performance. In addition, the choice of snapshot size appears to have a more pronounced impact on TADDY's performance than RustGraph's. On the other hand, RustGraph remains robust and consistently performs well across various snapshot sizes, demonstrating its robustness to different temporal resolutions.

F. Graph Properties Analysis

In this section, we examine the time-variant graph properties to understand the improvements demonstrated by RustGraph compared to other baselines. We visualize the graph clustering

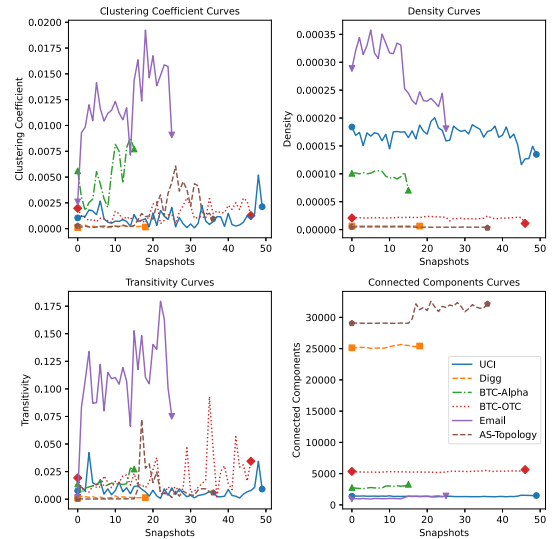


Fig. 9. Evolution of four dynamic graph properties over time.

coefficient [64], density, transitivity, and number of connected components as the graph evolves, as shown in Fig. 9.

The first three subplots in Fig. 9, the clustering coefficient, density, and transitivity metrics, reflect the graph's connectivity, where larger values suggest a higher level of connectivity or information flow. Notably, Email-DNC exhibits a higher mean and greater fluctuation compared to other datasets, indicating a richer source of structural-temporal information. This may explain the more substantial performance improvement of RustGraph on this dataset.

The final subplot, connected components, reflects the number of disconnected cliques in the dataset. It's worth noting that typical GNN-based methods tend to perform better on datasets with fewer connected components because they make reliable detections in a more unified and interconnected graph. This is corroborated by the fact that the two most challenging datasets (AS-Topology and Digg), as indicated by the curves, also exhibit the lowest average performance in Table II (calculated

by averaging the AUC results of all listed methods except RustGraph). Interestingly, RustGraph still achieved considerable performance on these two challenging datasets. We speculate that this counterintuitive phenomenon could be because the proposed encoder has a strong capability to capture local structural information within connected components through learning with reconstruction. Additionally, the robust anomaly detector helps reduce noise caused by large number of connected components. Therefore, given that other models struggle to learn effective features on challenging datasets, RustGraph's performance remains robust.

VI. CONCLUSION AND FUTURE WORK

In this paper, we first analyze two major limitations of recent studies for anomaly detection in dynamic graphs: the separately modeling of structural and temporal information in dynamic graph representation learning; and the vulnerable models trained by unreliable noisy labels. To overcome these obstacles, we propose RustGraph, a robust anomaly detection framework that jointly learns structural-temporal dependency of dynamic graphs, which is also a self-supervised framework that can be trained end-to-end. Specifically, we leverage a graph-based variational auto-encoder and a temporal consistent graph contrastive learning module to simultaneously encode structural and temporal dependencies, and a robust anomaly detector that secures the data reliability and model robustness. Our proposed method achieves state-of-the-art performance on six real-world dynamic graph datasets. Despite the success of the model, there are still areas for improvement. For example, in dynamic graphs, the time interval between snapshots is an important parameter, but the current works have rarely considered it, and we may leave it as future work.

REFERENCES

- [1] L. Wu, P. Cui, J. Pei, L. Zhao, and L. Song, "Graph neural networks," in *Graph Neural Networks: Foundations, Frontiers, and Applications*. Berlin, Germany: Springer, 2022, pp. 27–37.
- [2] S. M. Kazemi, "Dynamic graph neural networks," in *Graph Neural Networks: Foundations, Frontiers, and Applications*. Berlin, Germany: Springer, 2022, pp. 323–349.
- [3] T. Y. Berger-Wolf and J. Saia, "A framework for analysis of dynamic social networks," in *Proc. 12th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2006, pp. 523–528.
- [4] K. Zhao, Z. Zhang, Y. Rong, J. X. Yu, and J. Huang, "Finding critical users in social communities via graph convolutions," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 456–468, Jan. 2023.
- [5] Y. Zheng et al., "Clustering social audiences in business information networks," *Pattern Recognit.*, vol. 100, 2020, Art. no. 107126.
- [6] M. Zhang, S. Wu, X. Yu, Q. Liu, and L. Wang, "Dynamic graph neural networks for sequential recommendation," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 4741–4753, May 2022.
- [7] A. F. Subahi and K. E. Bouazza, "An intelligent IoT-based system design for controlling and monitoring greenhouse temperature," *IEEE Access*, vol. 8, pp. 125 488–125 500, 2020.
- [8] Z. Cheng et al., "Time2Graph+: Bridging time series and graph representation learning via multiple attentions," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 2078–2090, Feb. 2023.
- [9] X. Ma et al., "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, early access, Oct 8, 2021, doi: [10.1109/TKDE.2021.3118815](https://doi.org/10.1109/TKDE.2021.3118815).
- [10] W. Cheng, K. Zhang, H. Chen, G. Jiang, Z. Chen, and W. Wang, "Ranking causal anomalies via temporal and dynamical analysis on vanishing correlations," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, San Francisco, CA, USA, 2016, pp. 805–814, doi: [10.1145/2939672.2939765](https://doi.org/10.1145/2939672.2939765).
- [11] B. Dong et al., "Efficient discovery of abnormal event sequences in enterprise security systems," in *Proc. ACM Conf. Inf. Knowl. Manage.*, Singapore, 2017, pp. 707–715, doi: [10.1145/3132847.3132854](https://doi.org/10.1145/3132847.3132854).
- [12] S. Ranshous, S. Shen, D. Koutra, S. Harenberg, C. Faloutsos, and N. F. Samatova, "Anomaly detection in dynamic networks: A survey," *Wiley Interdiscipl. Rev.: Comput. Statist.*, vol. 7, no. 3, pp. 223–247, 2015.
- [13] W. Yu, W. Cheng, C. C. Aggarwal, K. Zhang, H. Chen, and W. Wang, "NetWalk: A flexible deep embedding approach for anomaly detection in dynamic networks," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, London, U.K., 2018, pp. 2672–2681, doi: [10.1145/3219819.3220024](https://doi.org/10.1145/3219819.3220024).
- [14] L. Zheng, Z. Li, J. Li, Z. Li, and J. Gao, "AddGraph: Anomaly detection in dynamic graph using attention-based temporal GCN," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Macao, China, 2019, pp. 4419–4425, doi: [10.24963/ijcai.2019/614](https://doi.org/10.24963/ijcai.2019/614).
- [15] L. Cai et al., "Structural temporal graph neural networks for anomaly detection in dynamic graphs," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 3747–3756.
- [16] Y. Liu et al., "Anomaly detection in dynamic graphs via transformer," *IEEE Trans. Knowl. Data Eng.*, early access, Nov. 2, 2021, doi: [10.1109/TKDE.2021.3124061](https://doi.org/10.1109/TKDE.2021.3124061).
- [17] Y. Yang, Y. Xu, Y. Sun, Y. Dong, F. Wu, and Y. Zhuang, "Mining fraudsters and fraudulent strategies in large-scale mobile social networks," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 1, pp. 169–179, Jan. 2021.
- [18] S. M. Kazemi et al., "Representation learning for dynamic graphs: A survey," *J. Mach. Learn. Res.*, vol. 21, no. 1, pp. 2648–2720, 2020.
- [19] S. Wang and P. S. Yu, "Graph neural networks in anomaly detection," in *Graph Neural Networks: Foundations, Frontiers, and Applications*. Berlin, Germany: Springer, 2022, pp. 557–578.
- [20] C. C. Aggarwal, Y. Zhao, and S. Y. Philip, "Outlier detection in graph streams," in *Proc. IEEE 27th Int. Conf. Data Eng.*, 2011, pp. 399–409.
- [21] S. Ranshous, S. Harenberg, K. Sharma, and N. F. Samatova, "A scalable approach for outlier detection in edge streams using sketch-based approximations," in *Proc. SIAM Int. Conf. Data Mining*, Miami, Florida, USA, 2016, pp. 189–197, doi: [10.1137/1.9781611974348.22](https://doi.org/10.1137/1.9781611974348.22).
- [22] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Representations*, Toulon, France, 2017. [Online]. Available: <https://openreview.net/forum?id=SJU4ayYgl>
- [23] R. Liao, "Graph neural networks: Graph generation," in *Graph Neural Networks: Foundations, Frontiers, and Applications*. Berlin, Germany: Springer, 2022, pp. 225–250.
- [24] T. Kipf and M. Welling, "Variational graph auto-encoders," 2016, *arXiv:1611.07308*.
- [25] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Annu. Conf. Neural Inf. Process. Syst.*, Long Beach, CA, USA, 2017, pp. 1024–1034. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7e9ea9-Abstract.html>
- [26] E. Hajiramezanali, A. Hasanazadeh, K. R. Narayanan, N. Duffield, M. Zhou, and X. Qian, "Variational graph recurrent neural networks," in *Proc. Annu. Conf. Neural Inf. Process. Syst.*, Vancouver, BC, Canada, 2019, pp. 10 700–10 710. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/a6b8deb7798e7532ade2a8934477d3ce-Abstract.html>
- [27] D. Kreuzer, D. Beaini, W. Hamilton, V. Létourneau, and P. Tossou, "Re-thinking graph transformers with spectral attention," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 21618–21629.
- [28] X. Liu et al., "Self-supervised learning: Generative or contrastive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023.
- [29] Y. Wang, W. Jin, and T. Derr, "Graph neural networks: Self-supervised learning," in *Graph Neural Networks: Foundations, Frontiers, and Applications*. Berlin, Germany: Springer, 2022, pp. 391–420.
- [30] P. Velickovic, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep graph infomax," in *Proc. 7th Int. Conf. Learn. Representations*, New Orleans, LA, USA, 2019. [Online]. Available: <https://openreview.net/forum?id=rklz9iAcKQ>

- [31] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 5812–5823.
- [32] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Deep graph contrastive representation learning," 2020, *arXiv:2006.04131*.
- [33] B. Chen et al., "GCCAD: Graph contrastive learning for anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 8, pp. 8037–8051, Aug. 2023.
- [34] S. Tian, R. Wu, L. Shi, L. Zhu, and T. Xiong, "Self-supervised representation learning on dynamic graphs," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 1814–1823.
- [35] L. Wang et al., "TCL: Transformer-based dynamic graph modelling via contrastive learning," 2021, *arXiv:2105.07944*.
- [36] S. Sukhbaatar, J. Bruna, M. Paluri, L. Bourdev, and R. Fergus, "Training convolutional networks with noisy labels," in *Proc. 3rd Int. Conf. Learn. Representations Workshop*, 2015.
- [37] J. Goldberger and E. Ben-Reuven, "Training deep neural-networks using a noise adaptation layer," in *Proc. 5th Int. Conf. Learn. Representations*, Toulon, France, 2017. [Online]. Available: <https://openreview.net/forum?id=H12GRgcxg>
- [38] T. Miyato, S. Ichi Maeda, M. Koyama, and S. Ishii, "Virtual adversarial training: A regularization method for supervised and semi-supervised learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 8, pp. 1979–1993, Aug. 2019.
- [39] S. Reed, H. Lee, D. Anguelov, C. Szegedy, D. Erhan, and A. Rabinovich, "Training deep neural networks on noisy labels with bootstrapping," in *Proc. 3rd Int. Conf. Learn. Representations Workshop*, 2015. [Online]. Available: <http://arxiv.org/abs/1412.6596>
- [40] J. Jin, Y. Li, and C. L. P. Chen, "Pattern classification with corrupted labeling via robust broad learning system," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 10, pp. 4959–4971, Oct. 2022.
- [41] J. Li et al., "Compositional temporal grounding with structured variational cross-graph correspondence learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 3032–3041.
- [42] J. Li et al., "Variational cross-graph reasoning and adaptive structured semantics learning for compositional temporal grounding," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 10, pp. 12601–12617, Oct. 2023.
- [43] T. Liu and D. Tao, "Classification with noisy labels by importance reweighting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 3, pp. 447–461, Mar. 2015.
- [44] Y. Wang, A. Kucukelbir, and D. M. Blei, "Robust probabilistic modeling with Bayesian data reweighting," in *Proc. 34th Int. Conf. Mach. Learn.*, Sydney, NSW, Australia, 2017, pp. 3646–3655. [Online]. Available: <http://proceedings.mlr.press/v70/wang17g.html>
- [45] L. Jiang et al., "MentorNet: Learning data-driven curriculum for very deep neural networks on corrupted labels," in *Proc. 35th Int. Conf. Mach. Learn.*, Stockholm, Sweden, 2018, pp. 2309–2318. [Online]. Available: <http://proceedings.mlr.press/v80/jiang18c.html>
- [46] B. Han et al., "Co-teaching: Robust training of deep neural networks with extremely noisy labels," in *Proc. Annu. Conf. Neural Inf. Process. Syst.*, Montréal, Canada, 2018, pp. 8536–8546. [Online]. Available: <https://proceedings.neurips.cc/paper/2018/hash/a19744e268754fb0148b017647355b7b-Abstract.html>
- [47] E. Dai, C. Aggarwal, and S. Wang, "NRGNN: Learning a label noise resistant graph neural network on sparsely and noisily labeled graphs," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, New York, NY, USA: Association for Computing Machinery, 2021, pp. 227–236, doi: [10.1145/3447548.3467364](https://doi.org/10.1145/3447548.3467364).
- [48] J. Xu et al., "Robust network enhancement from flawed networks," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 7, pp. 3507–3520, Jul. 2022.
- [49] M. Zhang, L. Hu, C. Shi, and X. Wang, "Adversarial label-flipping attack and defense for graph neural networks," in *Proc. IEEE Int. Conf. Data Mining*, 2020, pp. 791–800.
- [50] Y. Li, J. Yin, and L. Chen, "Unified robust training for graph neural networks against label noise," in *Proc. 25th Pacific-Asia Conf.*, 2021, pp. 528–540, doi: [10.1007/978-3-030-75762-5_42](https://doi.org/10.1007/978-3-030-75762-5_42).
- [51] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *Proc. 2nd Int. Conf. Learn. Representations*, Banff, AB, Canada, 2014. [Online]. Available: <http://arxiv.org/abs/1312.6114>
- [52] T. Chen, S. Kornblith, M. Norouzi, and G. E. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607. [Online]. Available: <http://proceedings.mlr.press/v119/chen20j.html>
- [53] M. Rotman and L. Wolf, "Shuffling recurrent neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 9428–9435.
- [54] T. Opsahl and P. Panzarasa, "Clustering in weighted networks," *Social Netw.*, vol. 31, no. 2, pp. 155–163, 2009.
- [55] M. De Choudhury, H. Sundaram, A. John, and D. D. Seligmann, "Social synchrony: Predicting mimicry of user actions in online social media," in *Proc. Int. Conf. Comput. Sci. Eng.*, 2009, pp. 151–158.
- [56] R. A. Rossi and N. K. Ahmed, "The network data repository with interactive graph analytics and visualization," in *Proc. 29th AAAI Conf. Artif. Intell.*, Austin, Texas, USA, 2015, pp. 4292–4293. [Online]. Available: <http://www.aaai.org/ocs/index.php/AAAI/AAAI15/paper/view/9553>
- [57] S. Kumar, F. Spezzano, V. Subrahmanian, and C. Faloutsos, "Edge weight prediction in weighted signed networks," in *Proc. IEEE 16th Int. Conf. Data Mining*, 2016, pp. 221–230.
- [58] S. Kumar, B. Hooi, D. Makhija, M. Kumar, C. Faloutsos, and V. S. Subrahmanian, "REV2: Fraudulent user prediction in rating platforms," in *Proc. 11th ACM Int. Conf. Web Search Data Mining*, Marina Del Rey, CA, USA, 2018, pp. 333–341, doi: [10.1145/3159652.3159729](https://doi.org/10.1145/3159652.3159729).
- [59] B. Zhang, R. Liu, D. Massey, and L. Zhang, "Collecting the internet as-level topology," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 35, no. 1, pp. 53–61, 2005.
- [60] D. Eswaran and C. Faloutsos, "SedanSpot: Detecting anomalies in edge streams," in *Proc. IEEE Int. Conf. Data Mining*, 2018, pp. 953–958.
- [61] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, San Francisco, CA, USA, 2016, pp. 855–864, doi: [10.1145/2939672.2939754](https://doi.org/10.1145/2939672.2939754).
- [62] U. Von Luxburg, "A tutorial on spectral clustering," *Statist. Comput.*, vol. 17, no. 4, pp. 395–416, 2007.
- [63] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA, 2014, pp. 701–710, doi: [10.1145/2623330.2623732](https://doi.org/10.1145/2623330.2623732).
- [64] J. Saramäki, M. Kivelä, J.-P. Onnela, K. Kaski, and J. Kertész, "Generalizations of the clustering coefficient to weighted complex networks," *Phys. Rev. E*, vol. 75, Feb. 2007, Art. no. 027105, doi: [10.1103/PhysRevE.75.027105](https://doi.org/10.1103/PhysRevE.75.027105).



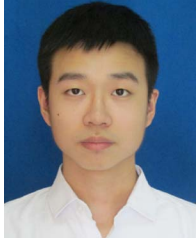
Jianhao Guo received the BE degree from Zhejiang University, Hangzhou, China, in 2020. He is currently working toward the PhD degree in computer science with Zhejiang University, under the supervision of Prof. Siliang Tang. His research interests include concentrates on graph learning and its applications. He has published a paper focusing the problem of graph contrastive learning in the top-tier conference IJCAI as the first co-author.



Siliang Tang (Member, IEEE) received the PhD degree from the Nation University of Ireland, Maynooth. He is currently a full professor with the College of Computer Science, Zhejiang University. His research interests include natural language processing, cross-media analysis, and graph mining. So far, he has published more than a hundred papers in the top-tier scientific conferences/journals such as AAAI, IJCAI, KDD, NIPS, ICML, ACL, EMNLP, NAACL, SIGIR, *IEEE Transactions on Knowledge and Data Engineering* (NLP and Graph mining); *ACM MM*, *CVPR*, *ICCV*, *IEEE Transactions on Multimedia*, *IEEE Transactions on Image Processing*, *IEEE Transactions on Circuits and Systems for Video* (Multimodal Understanding, and Graph Neural Networks in Computer Vision); *IEEE VIS*, *IEEE Transactions on Visualization and Computer Graphics* (Data Visualization). He has been serving as an area chair/senior PC member or PC member in conferences such as AAAI, IJCAI, NIPS, ICML, CVPR, ACL, EMNLP, NAACL, and reviewers of journals such as *IEEE Transactions on Image Processing*, *IEEE Transactions on Multimedia*, *IEEE Transactions on Systems, Man, and Cybernetics*, *ACM Computing Surveys*, *Nature-Scientific Reports*, etc.



Juncheng Li received the BE degree from the Harbin Institute of Technology, China, in 2019. He is currently working toward the PhD degree with Zhejiang University, advised by Prof. Yueting Zhuang and Prof. Siliang Tang. His research interests include vision-and-language, video understanding, embodied intelligence, and graph mining. He has published several papers in top-tier scientific conferences as the first author, such as CVPR, ICCV, NeurIPS, ACM Multimedia.



Kaihang Pan received the BE degree in software engineering from Zhejiang University, Hangzhou, China, in 2018. He is currently working toward the MS degree in computer science with Zhejiang University, Hangzhou, China. His research concentration includes graph data mining and natural language processing.



Lingfei Wu received the PhD degree in computer science from the College of William and Mary, in 2016. He is an engineering manager with the Content and Knowledge Graph Group at Pinterest, where he focuses on building the next generation Knowledge Graph to empower Pinterest recommendation/research systems. He has published more than 70 top-ranked conference and journal papers and is a co-inventor of more than 30 filed US patents. He was the recipients of the Best Paper Award and Best Student Paper Award of several conferences such as

AAAI workshop on DLGMA'20, IEEE ICC'19 and KDD workshop on DLG'19. His research has been featured in numerous media outlets, including NatureNews, YahooNews, Venturebeat, and TechTalks. He has co-organized more than ten conferences and workshops including AAAI'21, IEEE BigData'19, IEEE BigData'18 and is the founding co-chair for Workshops of Deep Learning on Graphs (with KDD'20, AAAI'20, KDD'19, and IEEE BigData'19). He has currently served as associate editor of *ACM Transactions on Knowledge Discovery from Data* and *International Journal of Intelligent Systems*, and regularly served as a SPC/PC member of the following major AI/ML/NLP conferences including KDD, IJCAI, AAAI, NIPS, ICML, ICLR, and ACL.